

DOCUMENTAÇÃO DO PROJETO

Feedback Platform

Tech Challenge — Fase 4
Cloud Computing, Serverless e Deploy

Plataforma serverless para recebimento, armazenamento e análise de
feedbacks de estudantes.

Tecnologias principais: Java 25 • Azure Functions • Azure Table Storage • Azure Queue Storage • Azure
Communication Services • Application Insights • GitHub Actions

1. Visão geral

O Feedback Platform é uma aplicação em nuvem que recebe avaliações de estudantes, persiste os feedbacks, envia alertas por e-mail quando uma avaliação é crítica e gera um relatório periódico para apoiar o acompanhamento da satisfação dos alunos.

O projeto foi desenvolvido para atender ao Tech Challenge da Fase 4, cujo enunciado exige aplicação cloud com funções serverless, persistência, notificações críticas, relatório semanal, monitoramento, segurança, deploy automatizado, código-fonte público e vídeo de demonstração.

1.1 Objetivos

- Receber feedbacks por uma API HTTP.
- Validar descrição e nota entre 0 e 10.
- Classificar a urgência conforme a nota.
- Armazenar feedbacks no Azure Table Storage.
- Enfileirar notificações críticas e enviá-las por e-mail.
- Gerar e enviar relatório semanal por e-mail.
- Permitir implantação automatizada por GitHub Actions e acompanhar a execução via Application Insights.

2. Arquitetura da solução

A solução separa as responsabilidades em três Azure Functions. Os serviços de armazenamento, mensageria, e-mail e monitoramento são gerenciados na Azure.

Componente	Responsabilidade
ReceberFeedbackFunction HTTP Trigger	Recebe POST /api/avaliacao, valida o payload, calcula urgência, persiste o feedback e coloca alertas críticos na fila.
NotificarUrgenciaFunction Queue Trigger	Consome mensagens da fila notificacoes-urgencia e envia e-mail ao administrador para feedbacks críticos.
GerarRelatorioFunction Timer Trigger	Executa em agenda configurada, consulta feedbacks recentes, consolida indicadores e envia o relatório por e-mail.
Azure Table Storage	Armazena os feedbacks na tabela Feedbacks.
Azure Queue Storage	Desacopla o recebimento de feedbacks críticos do envio de e-mail.
Azure Communication Services Email	Entrega e-mails de alerta e relatório.
Application Insights	Coleta telemetria para acompanhar requisições e execuções.
GitHub Actions	Automatiza build e deploy quando há push para master.

2.1 Fluxo de recebimento

1. O cliente envia POST /api/avaliacao com descricao e nota.
2. A Function valida os dados e classifica a urgência.
3. O feedback é persistido na tabela Feedbacks.
4. Se a urgência for CRITICA, uma mensagem é enviada à fila notificacoes-urgencia.
5. A Function de fila processa a mensagem e envia o e-mail ao administrador.

3. API HTTP

Endpoint publicado: POST <https://feedback-platform-functions-2026-g9hdefh7gre9hmae.brazilsouth-01.azurewebsites.net/api/avaliacao>

A função está configurada com autenticação FUNCTION. As chamadas devem incluir uma chave válida no parâmetro code ou no cabeçalho x-functions-key. Não publique a chave no repositório, no README ou em capturas de tela.

3.1 Exemplo de corpo JSON

```
{
  "descricao": "A aula foi excelente",
  "nota": 9
}
```

3.2 Validações e classificação

Nota	Urgência
0 a 4	CRITICA
5 a 7	MODERADA
8 a 10	NORMAL

A descrição deve ser preenchida e a nota deve ser um número inteiro de 0 a 10. O comportamento esperado para dados inválidos é uma resposta de erro de validação.

4. Persistência e mensageria

Os feedbacks são armazenados na tabela Feedbacks, com PartitionKey FEEDBACK e RowKey identificador UUID. Entre os atributos persistidos estão descrição, nota, data de envio e urgência.

A fila notificacoes-urgencia desacopla a API do processo de envio de e-mail. Isso permite que a Function HTTP responda sem precisar concluir a entrega do e-mail na mesma execução. Falhas no processamento da mensagem podem ser observadas e tratadas pelo mecanismo de retry da fila.

5. E-mails e relatório semanal

A NotificarUrgenciaFunction envia um e-mail para o administrador quando recebe uma mensagem referente a um feedback crítico. O aviso inclui descrição, urgência e data de envio, conforme o enunciado.

A GerarRelatorioFunction usa um Timer Trigger configurado para executar às 11:00 UTC nas segundas-feiras (08:00 no horário de Maceió, UTC−3). O relatório consolida dados da janela recente de sete dias.

O relatório esperado deve apresentar a média das avaliações, quantidade de avaliações por dia e quantidade por urgência. O PDF do desafio também menciona descrição, urgência e data de envio como dados do relatório. Confirme esses campos na versão efetivamente enviada por e-mail antes da entrega.

6. Deploy automatizado (CI/CD)

O workflow está no arquivo `.github/workflows/master_feedback-platform-functions-2026.yml`. O gatilho push para a branch master inicia a automação; `workflow_dispatch` permite execução manual.

Etapas principais:

- Checkout do repositório.
- Configuração do JDK Temurin 25 por actions/setup-java@v4.
- Compilação e empacotamento com mvn clean package.
- Login na Azure com azure/login@v2 e GitHub Secrets.
- Publicação por Azure/functions-action@v1 na Function App feedback-platform-functions-2026.

A execução observada no GitHub Actions concluiu com sucesso. Portanto, novos commits enviados para master iniciam o fluxo automaticamente; a publicação só deve ser considerada concluída quando o workflow terminar sem falhas.

7. Monitoramento

A Function App está integrada ao Application Insights. A telemetria permite acompanhar requisições, duração e execuções das Functions. A consulta de logs pode ser feita no portal Azure, na área de monitoramento da aplicação.

Para fortalecer a demonstração, registre no vídeo uma execução bem-sucedida e mostre onde consultar logs e falhas. O requisito de aplicação monitorada não significa, por si só, que alertas personalizados já estejam configurados; confirme se existem regras de alerta específicas antes de afirmar isso.

8. Segurança e governança

- A Function HTTP usa autenticação FUNCTION; a chave não deve ser divulgada.
- A opção Somente HTTPS foi ativada na configuração da Function App.
- A opção de acesso anônimo ao Blob Storage está desativada.
- O acesso por chave da conta de armazenamento está habilitado porque a aplicação usa connection strings; desativá-lo sem migrar a autenticação pode interromper o funcionamento.
- As connection strings devem ficar em configurações da aplicação/secrets e nunca no código-fonte ou em commits.
- O .gitignore inclui local.settings.json, bin/ e obj/. O histórico do repositório foi conferido pelo usuário e não foi encontrado local.settings.json.

Melhoria futura: migrar acessos ao Storage para Managed Identity e RBAC de menor privilégio. Para a entrega acadêmica, documente a configuração atual com transparência e não afirme que Managed Identity já está implementada.

9. Como executar os testes com Postman

- Importe a coleção Feedback Platform - Azure Functions.
- Configure baseUrl com a URL da Function App e functionKey com a chave atual, mantendo a chave local.
- Execute o teste normal com nota 9; confira urgência NORMAL.
- Execute o teste crítico com nota 2; confirme o recebimento do e-mail.
- Execute os testes de nota inválida e descrição vazia; confirme as respostas de validação.

A requisição crítica envia um e-mail real ao administrador. Use-a de forma consciente durante a demonstração.

10. Roteiro sugerido para o vídeo

- Apresentar o objetivo e a arquitetura.
- Mostrar a Function App ativa e as três Functions no portal Azure.
- Enviar um feedback normal pelo Postman e mostrar a resposta.
- Enviar um feedback crítico e mostrar o e-mail de alerta.
- Mostrar a tabela Feedbacks no Storage.
- Apresentar o relatório semanal recebido por e-mail e seus indicadores.
- Mostrar o Application Insights e a telemetria.
- Mostrar o workflow GitHub Actions com execução concluída com sucesso.
- Explicar as configurações de segurança sem exibir chaves ou connection strings.

11. Checklist de entrega

- Repositório público com código-fonte.
- README.md com arquitetura, deploy, monitoramento e descrição das Functions.
- Workflow de CI/CD executando build e deploy com sucesso.
- Testes demonstrados para feedback normal, crítico e dados inválidos.
- E-mail crítico confirmado.
- Relatório semanal revisado para garantir todos os campos exigidos.
- Application Insights demonstrado.
- Vídeo gravado mostrando aplicação, Functions e configurações.
- Nenhuma chave, connection string ou credencial incluída no repositório ou no vídeo.

Observação de escopo

Este documento descreve a arquitetura e as configurações registradas durante a implementação e os testes relatados. Os campos exatos do relatório, regras de alerta personalizadas, permissões detalhadas do Storage e a execução integral de todos os cenários devem ser validados no ambiente antes da entrega. Nunca inclua valores reais de chaves ou connection strings nesta documentação.